STATE DIAGRAMS

EduHub | Online Learning Platform

14	STATE MACHINES	08	STORED STATUS MODELS	03	APPLICATION ROLES

Scope: UML state machine views, event-triggered transitions, guards, persistence mapping, state invariants and test scenarios.

Based on the EduHub brief and approved candidate architecture / ERD / logical schema baseline. Additional policies remain proposed.

Model purpose & boundaries

A UML state machine focuses on the lifecycle of a single object: its permissible states, the events that move it between states, and the conditions and effects attached to those transitions.

14 **Lifecycle diagrams**

Persistent and derived views

08 **Enum-backed models**

Exact schema-aligned values

06 **Non-enum models**

Flags, timestamps, computed, transient

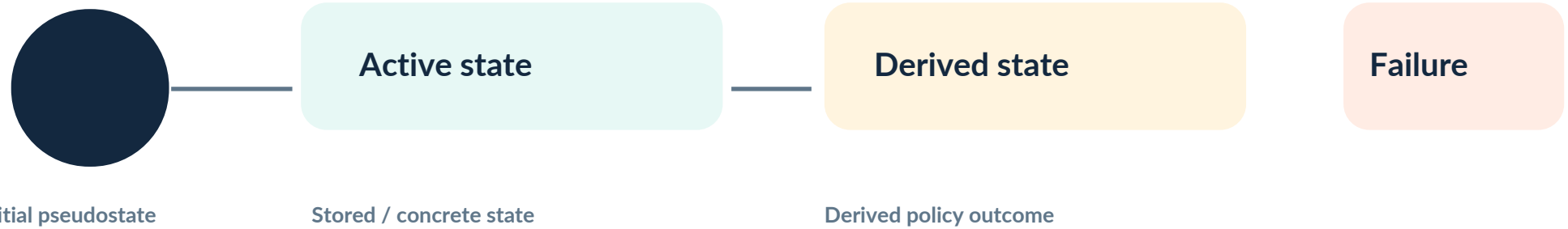
IN SCOPE

Courses and media; enrollments and learning progress; assessments; orders and Stripe events; discounts; certificates; learner engagement; cart and coupon policy evaluation.

OUT OF SCOPE

Detailed database attributes, REST payloads, UI screens, full operational process models, infrastructure provisioning and claims that the project has already been implemented.

UML state notation & interpretation



TRANSITION NOTATION

triggerEvent [guard expression] / transition effect

Trigger	The occurrence or call that requests a state change.
Guard	A server-evaluated condition that must be true before transition.
Effect	An observable or persisted action that occurs on the transition.
Self-transition	The event does not change the named lifecycle status.
Initial	An initial arrow identifies creation/first conceptual state; not a user event.
Derived	Label is computed; it must NOT be added as an enum value.

State-machine catalog

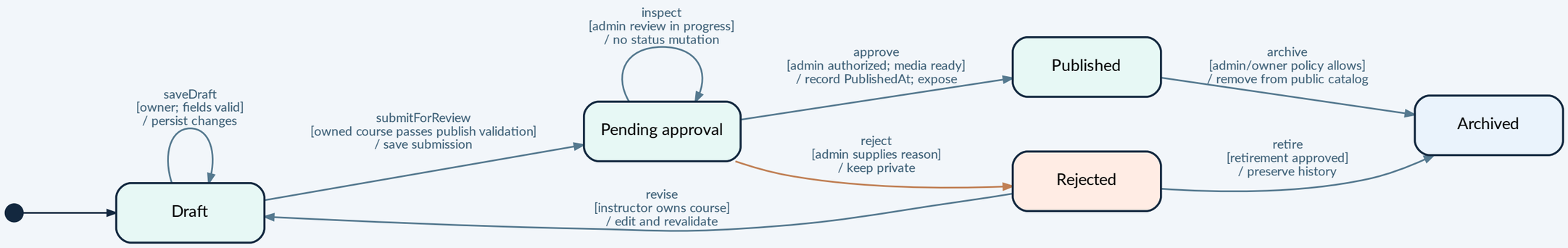
Eight diagrams reproduce an existing SQL status enum; six use flags, row existence, or policy-level derived states.

ID	Lifecycle	Representation	Data owner
SM-01	Course approval and publication	Stored enum	Courses
SM-02	Video and media processing	Stored enum	MediaAssets
SM-03	Student enrollment entitlement	Stored enum	Enrollments
SM-04	Per-lesson learning progress	Derived state	LessonProgresses
SM-05	Quiz attempt and grading	Stored enum	QuizAttempts
SM-06	Order settlement lifecycle	Stored enum	Orders
SM-07	Stripe payment transaction	Stored enum	PaymentTransactions
SM-08	Stripe webhook event processing	Stored enum	PaymentWebhookEvents
SM-09	Coupon redemption accounting	Stored enum	CouponRedemptions
SM-10	Certificate issuance and revocation	Mixed / derived	Certificates
SM-11	Course Q&A resolution	Derived / flag	CourseQuestions
SM-12	In-app notification read status	Derived / timestamp	Notifications
SM-13	Shopping cart content lifecycle	Derived state	Carts / CartItems
SM-14	Coupon eligibility evaluation	Computed policy state	Coupons

MODEL STATUS / **Candidate design for team and supervisor approval**

Course approval and publication

SM-01 | CATALOG + GOVERNANCE | STORED ENUM | Courses



PERSISTENCE / STATE SOURCE

Courses.Status

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-01 | transition contract

One instructor-owned course progresses from a private draft through moderation to public visibility. The five modeled statuses match the existing Course table.

ENTITY / STATE REPRESENTATION

Courses | Courses.Status

TRACEABILITY

UC-25, UC-31, UC-37, UC-38 / FR-013, FR-014, FR-018, FR-023, FR-069, FR-070

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Draft -> PendingApproval	submitForReview	owned course passes publish validation	save submission
PendingApproval -> Published	approve	admin authorized; media ready	record PublishedAt; expose
PendingApproval -> Rejected	reject	admin supplies reason	keep private
Rejected -> Draft	revise	instructor owns course	edit and revalidate
Draft -> Draft	saveDraft	owner; fields valid	persist changes
Published -> Archived	archive	admin/owner policy allows	remove from public catalog
Rejected -> Archived	retire	retirement approved	preserve history
PendingApproval -> PendingApproval	inspect	admin review in progress	no status mutation

INVARIANTS

- Only Published courses appear in the public catalog.
- Approval must not bypass server-side content checks.
- Course.RowVersion protects concurrent instructor/admin edits.

EXCEPTIONS / HANDLING

Unapproved publish requests are denied. Rejected courses can be revised before resubmission. Published-course revisions need a separately approved versioning policy.

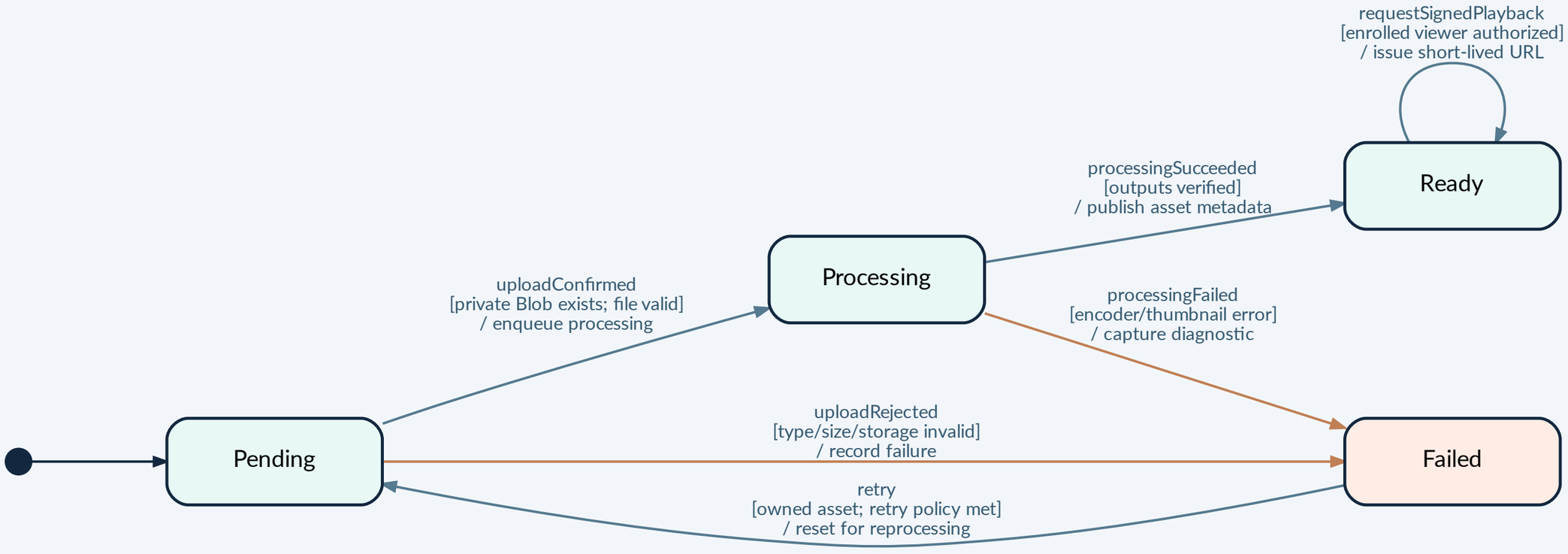
VERIFICATION / TEST

Instructor submits valid draft; admin approves; public listing appears. Verify rejected course remains private and stale RowVersion is rejected.

DESIGN NOTE: Submission and moderation are proposed workflow details; the brief also mentions immediate publish. Admin approval governs release in this model.

Video and media processing

SM-02 | MEDIA PIPELINE | STORED ENUM | MediaAssets



PERSISTENCE / STATE SOURCE

MediaAssets.ProcessingStatus

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-02 | transition contract

Private Azure Blob uploads are validated and processed before lessons receive playable media. Uploading is an action; the four mapped statuses are exactly those in the proposed schema.

ENTITY / STATE REPRESENTATION

MediaAssets | MediaAssets.ProcessingStatus

TRACEABILITY

UC-27, UC-28, UC-46, UC-47 / FR-024, FR-025, FR-026, FR-027, FR-028, FR-080

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Pending -> Processing	uploadConfirmed	private Blob exists; file valid	enqueue processing
Pending -> Failed	uploadRejected	type/size/storage invalid	record failure
Processing -> Ready	processingSucceeded	outputs verified	publish asset metadata
Processing -> Failed	processingFailed	encoder/thumbnail error	capture diagnostic
Failed -> Pending	retry	owned asset; retry policy met	reset for reprocessing
Ready -> Ready	requestSignedPlayback	enrolled viewer authorized	issue short-lived URL

INVARIANTS

- Signed URLs are generated on demand, never stored in BlobKey.
- Only Ready lesson media is available for secure playback.
- Failed upload or encoding does not publish the lesson video.

EXCEPTIONS / HANDLING

Transient Azure/storage or encoder failure retries under a bounded policy; repeated failure remains Failed for manual action.

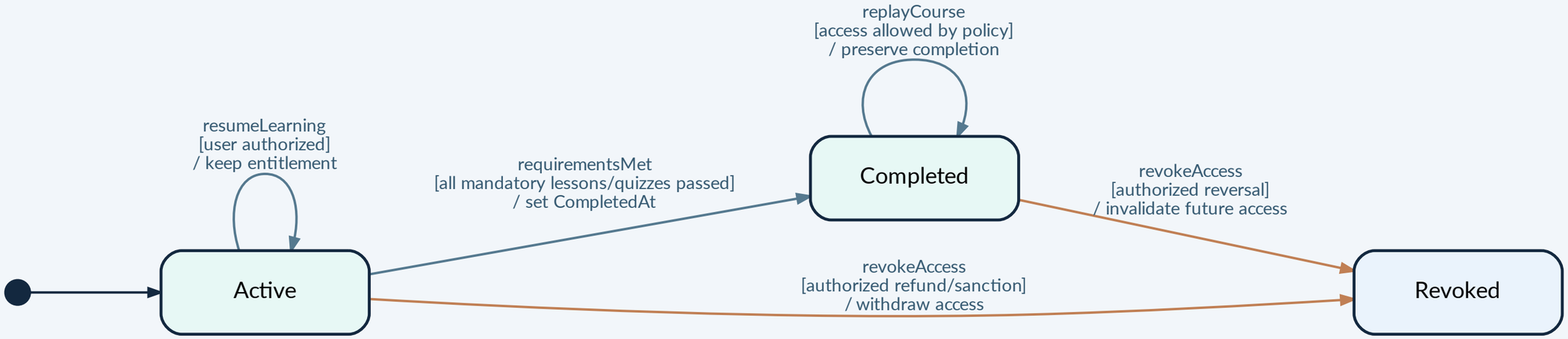
VERIFICATION / TEST

Upload valid video and verify Pending -> Processing -> Ready. Simulate processor failure and retry; ensure no public Blob URL leaks.

DESIGN NOTE: The worker/queue is recommended; no persistent Uploading state or separate transcoding job table is present.

Student enrollment entitlement

SM-03 | LEARNING + COMMERCE | STORED ENUM | Enrollments



PERSISTENCE / STATE SOURCE

Enrollments.Status

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-03 | transition contract

A student receives an Active enrollment for a free course or after confirmed paid fulfillment. Course completion and potential access revocation use the existing status values.

ENTITY / STATE REPRESENTATION

Enrollments | Enrollments.Status

TRACEABILITY

UC-07, UC-12, UC-44 / FR-030, FR-031, FR-032

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Active -> Completed	requirementsMet	all mandatory lessons/quizzes passed	set CompletedAt
Active -> Revoked	revokeAccess	authorized refund/sanction	withdraw access
Completed -> Revoked	revokeAccess	authorized reversal	invalidate future access
Active -> Active	resumeLearning	user authorized	keep entitlement
Completed -> Completed	replayCourse	access allowed by policy	preserve completion

INVARIANTS

- Unique(StudentUserId, CourseId) prevents duplicate enrollments.
- Paid Active enrollment only follows verified successful payment.
- CompletedAt is set when requirements are met; revocation is explicit.

EXCEPTIONS / HANDLING

Refund and re-purchase handling is a team policy decision because the uniqueness constraint prevents duplicate rows for the same student-course.

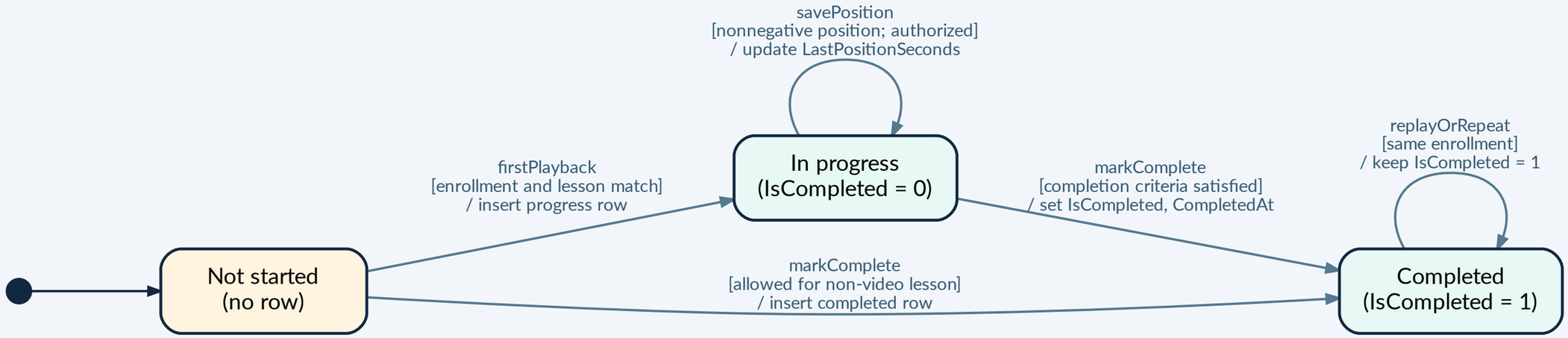
VERIFICATION / TEST

Free enrollment creates one Active row; replay is idempotent. Paid access is absent until webhook success; completion sets CompletedAt.

DESIGN NOTE: Revocation is a proposed policy represented by the existing Revoked status; reinstatement needs explicit requirements.

Per-lesson learning progress

SM-04 | LEARNING PROGRESS | DERIVED STATE | LessonProgresses



PERSISTENCE / STATE SOURCE

Row absence + IsCompleted + LastPositionSeconds

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-04 | transition contract

A lesson is conceptually Not started until a progress row exists, In progress while its row is incomplete, and Completed after an authorized completion event.

ENTITY / STATE REPRESENTATION

LessonProgresses | Row absence + IsCompleted + LastPositionSeconds

TRACEABILITY

UC-13, UC-14, UC-15 / FR-033, FR-034

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
NotStarted -> InProgress	firstPlayback	enrollment and lesson match	insert progress row
InProgress -> InProgress	savePosition	nonnegative position; authorized	update LastPositionSeconds
InProgress -> Completed	markComplete	completion criteria satisfied	set IsCompleted, CompletedAt
NotStarted -> Completed	markComplete	allowed for non-video lesson	insert completed row
Completed -> Completed	replayOrRepeat	same enrollment	keep IsCompleted = 1

INVARIANTS

- UNIQUE(EnrollmentId, LessonId) enforces one progress row.
- Lesson must belong to the enrolled course.
- A replay cannot silently revert Completed to In progress.

EXCEPTIONS / HANDLING

Out-of-order position updates should not decrease a learner position unless an explicit resume policy permits it; display percentage is derived.

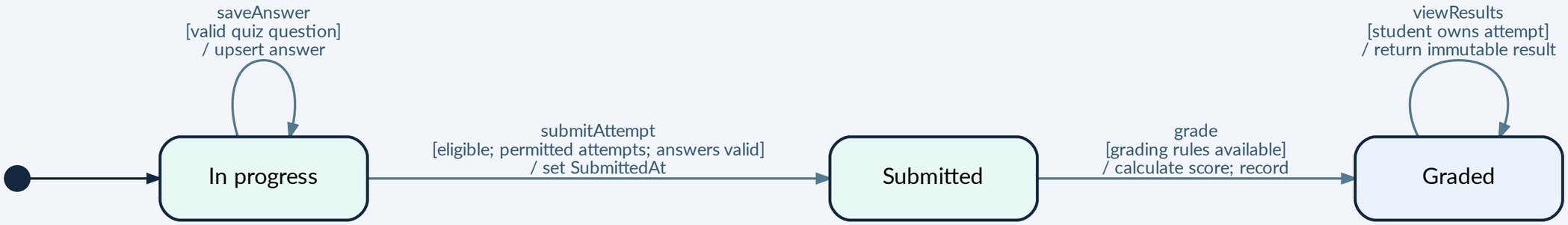
VERIFICATION / TEST

Run duplicate progress updates concurrently; one record remains. Complete a lesson then replay and verify completion is retained.

DESIGN NOTE: Not started is a conceptual state, not a stored enum; course completion is derived from underlying progress and quiz outcomes.

Quiz attempt and grading

SM-05 | ASSESSMENT | STORED ENUM | QuizAttempts



PERSISTENCE / STATE SOURCE

QuizAttempts.Status

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-05 | transition contract

A single quiz attempt records answers, submission, and grading. Pass/fail is computed from ScorePercentage against the quiz threshold, not encoded as a separate attempt status.

ENTITY / STATE REPRESENTATION

QuizAttempts | QuizAttempts.Status

TRACEABILITY

UC-16, UC-33 / FR-040, FR-041, FR-042, FR-043, FR-044

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
InProgress -> InProgress	saveAnswer	valid quiz question	upsert answer
InProgress -> Submitted	submitAttempt	eligible; permitted attempts; answers valid	set SubmittedAt
Submitted -> Graded	grade	grading rules available	calculate score; record
Graded -> Graded	viewResults	student owns attempt	return immutable result

INVARIANTS

- AttemptNumber unique per (QuizId, EnrollmentId).
- Submission is final for an attempt; retake creates a new attempt row.
- ScorePercentage is between 0 and 100; passed is derived.

EXCEPTIONS / HANDLING

Malformed answers, a mismatched enrollment, or repeated submit is rejected or idempotently acknowledged. Unfinished attempts remain InProgress in the current schema.

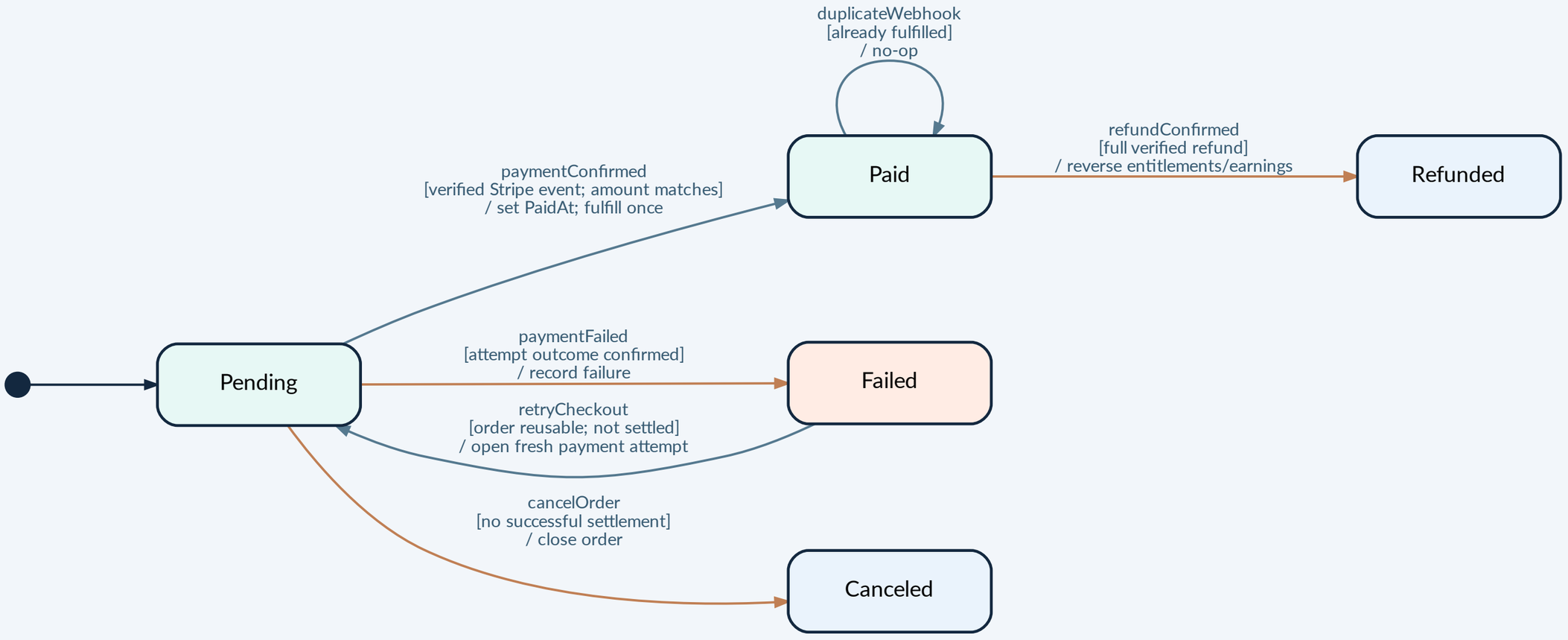
VERIFICATION / TEST

Submit valid and invalid quizzes; duplicate submit does not regrade twice; retake uses next AttemptNumber.

DESIGN NOTE: Timeout and abandoned states are not in the current table; an attempt-expiry policy would require separate rules or schema changes.

Order settlement lifecycle

SM-06 | COMMERCE | STORED ENUM | Orders



PERSISTENCE / STATE SOURCE

Orders.Status

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-06 | transition contract

An order captures immutable course and price snapshots. The order remains Pending until a server-verified payment outcome or an authorized cancellation is processed.

ENTITY / STATE REPRESENTATION

Orders | Orders.Status

TRACEABILITY

UC-10, UC-11, UC-43, UC-44 / FR-046, FR-050, FR-051, FR-053, FR-054, FR-055

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Pending -> Paid	paymentConfirmed	verified Stripe event; amount matches	set PaidAt; fulfill once
Pending -> Failed	paymentFailed	attempt outcome confirmed	record failure
Failed -> Pending	retryCheckout	order reusable; not settled	open fresh payment attempt
Pending -> Canceled	cancelOrder	no successful settlement	close order
Paid -> Refunded	refundConfirmed	full verified refund	reverse entitlements/earnings
Paid -> Paid	duplicateWebhook	already fulfilled	no-op

INVARIANTS

- Do not create an entitlement from the success redirect.
- OrderItem prices and order totals remain snapshots.
- The current enum supports full Refunded; partial refunds need separate modeling.

EXCEPTIONS / HANDLING

Late successful Stripe events after local cancellation require reconciliation; do not simply discard provider truth or double-enroll.

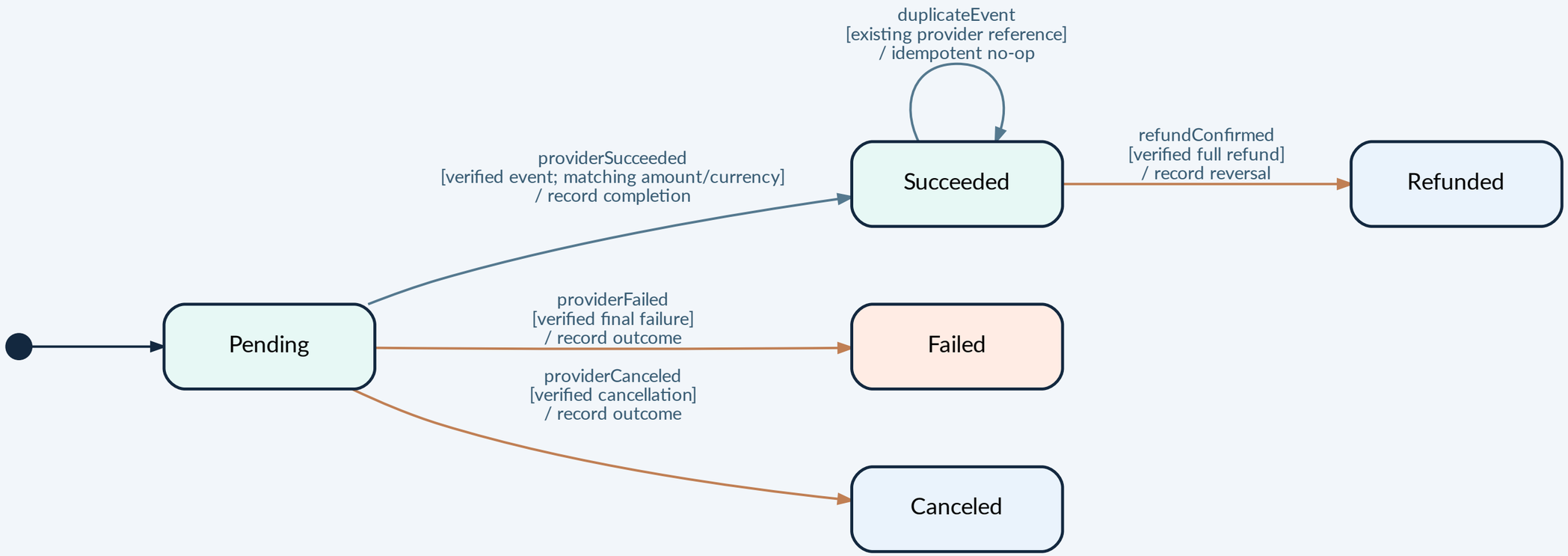
VERIFICATION / TEST

Confirm no enrollment on unverified redirect; webhook leads to one Paid order; duplicate webhook causes no duplicate records.

DESIGN NOTE: Retry on the same failed order and full-refund behavior are proposed policies; reconcile with the payment provider.

Stripe payment transaction

SM-07 | PAYMENT INTEGRATION | STORED ENUM | PaymentTransactions



PERSISTENCE / STATE SOURCE

PaymentTransactions.Status

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-07 | transition contract

Each Stripe payment attempt has its own status. An order may have multiple transactions, but a succeeded transaction must not be overwritten by a stale failure event.

ENTITY / STATE REPRESENTATION

PaymentTransactions | PaymentTransactions.Status

TRACEABILITY

UC-10, UC-43 / FR-050, FR-054, FR-055, FR-081

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Pending -> Succeeded	providerSucceeded	verified event; matching amount/currency	record completion
Pending -> Failed	providerFailed	verified final failure	record outcome
Pending -> Canceled	providerCanceled	verified cancellation	record outcome
Succeeded -> Refunded	refundConfirmed	verified full refund	record reversal
Succeeded -> Succeeded	duplicateEvent	existing provider reference	idempotent no-op

INVARIANTS

- Only verified provider events drive settlement.
- StripeObjectId is unique when present.
- A failed payment retry creates a new transaction rather than mutating history.

EXCEPTIONS / HANDLING

Out-of-order events are reconciled using provider object identity and known current provider status; a failed event cannot regress Succeeded.

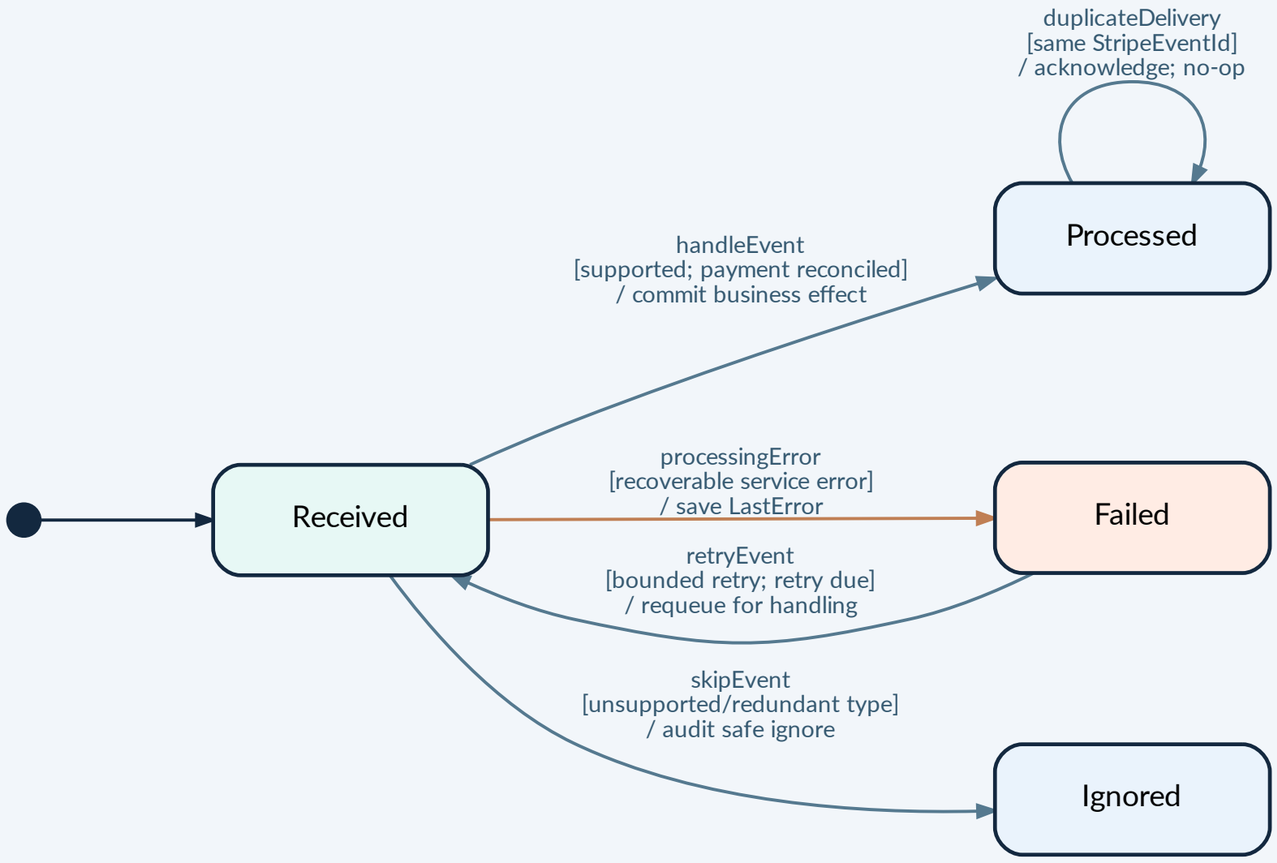
VERIFICATION / TEST

Simulate success, duplicate success, stale failure, cancellation, and refund; require monotonically correct financial state.

DESIGN NOTE: State machine is payment-attempt-level, distinct from the aggregate Order; partial refunds are not represented in the existing enum.

Stripe webhook event processing

SM-08 | PAYMENT RELIABILITY | STORED ENUM | PaymentWebhookEvents



PERSISTENCE / STATE SOURCE

PaymentWebhookEvents.Status

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-08 | transition contract

A webhook event is retained only after signature verification. Processing outcomes are recorded independently from the payment transaction and deduplicated by StripeEventId.

ENTITY / STATE REPRESENTATION

PaymentWebhookEvents | PaymentWebhookEvents.Status

TRACEABILITY

UC-43, UC-44 / FR-054, FR-055, FR-081

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Received -> Processed	handleEvent	supported; payment reconciled	commit business effect
Received -> Ignored	skipEvent	unsupported/redundant type	audit safe ignore
Received -> Failed	processingError	recoverable service error	save LastError
Failed -> Received	retryEvent	bounded retry; retry due	requeue for handling
Processed -> Processed	duplicateDelivery	same StripeEventId	acknowledge; no-op

INVARIANTS

- Invalid Stripe signatures never create a Received event.
- StripeEventId UNIQUE prevents double processing.
- Processing result and relevant business effects should commit atomically or be reconciled.

EXCEPTIONS / HANDLING

Retries must be safe after partial failure; an unsupported event is Ignored, not interpreted as payment success.

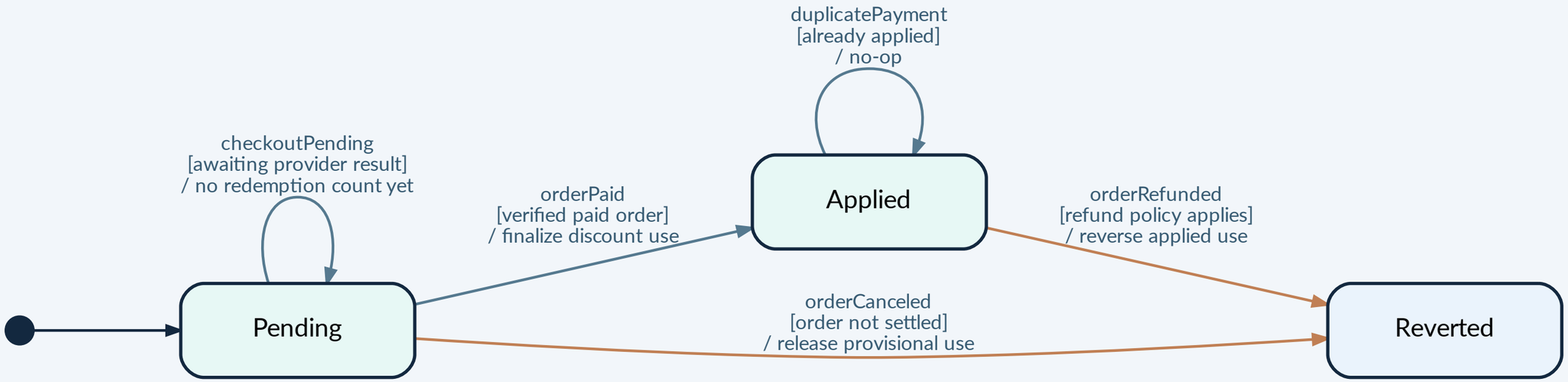
VERIFICATION / TEST

Send the same signed webhook repeatedly and assert one business effect. Send a bad signature and assert no persisted event.

DESIGN NOTE: Retry scheduling is recommended but not specified by the brief; no RetryCount column exists in the schema.

Coupon redemption accounting

SM-09 | PRICING + DISCOUNTS | STORED ENUM | CouponRedemptions



PERSISTENCE / STATE SOURCE

CouponRedemptions.Status

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-09 | transition contract

A pending redemption reserves an order-linked coupon use; it is counted as Applied only after settlement and can be Reverted after cancellation or refund.

ENTITY / STATE REPRESENTATION

CouponRedemptions | CouponRedemptions.Status

TRACEABILITY

UC-09, UC-10, UC-43 / FR-047, FR-048, FR-054, FR-055

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Pending -> Applied	orderPaid	verified paid order	finalize discount use
Pending -> Reverted	orderCanceled	order not settled	release provisional use
Applied -> Reverted	orderRefunded	refund policy applies	reverse applied use
Pending -> Pending	checkoutPending	awaiting provider result	no redemption count yet
Applied -> Applied	duplicatePayment	already applied	no-op

INVARIANTS

- UNIQUE(OrderId) prevents multiple coupon-redemption rows per order.
- Only Applied redemption rows count toward MaxRedemptions.
- AppliedAmount is an order-time discount snapshot.

EXCEPTIONS / HANDLING

If a coupon expires during an existing checkout, the order snapshot and reservation policy determine eligibility; never silently recalculate a settled order.

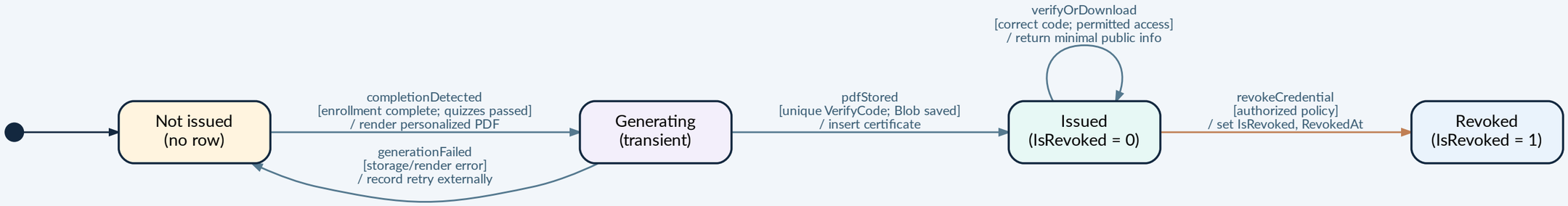
VERIFICATION / TEST

Apply coupon during checkout, fail payment, then retry; confirm limit counting only for Applied and no duplicate redemption.

DESIGN NOTE: Coupon reservation behavior is proposed; the brief requires coupon codes but does not specify how pending redemptions reserve capacity.

Certificate issuance and revocation

SM-10 | CERTIFICATION | MIXED / DERIVED | Certificates



PERSISTENCE / STATE SOURCE

Absent row -> persisted IsRevoked

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-10 | transition contract

Generation is transient work. The certificate row is created only when a private PDF and unique verification code are ready; IsRevoked tracks post-issue validity.

ENTITY / STATE REPRESENTATION

Certificates | Absent row -> persisted IsRevoked

TRACEABILITY

UC-05, UC-23, UC-24, UC-45 / FR-063, FR-064, FR-065, FR-066, FR-067, FR-068, FR-082

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Absent -> Generating	completionDetected	enrollment complete; quizzes passed	render personalized PDF
Generating -> Issued	pdfStored	unique VerifyCode; Blob saved	insert certificate
Generating -> Absent	generationFailed	storage/render error	record retry externally
Issued -> Issued	verifyOrDownload	correct code; permitted access	return minimal public info
Issued -> Revoked	revokeCredential	authorized policy	set IsRevoked, RevokedAt

INVARIANTS

- One certificate per EnrollmentId and unique VerifyCode.
- No certificate row is created until PdfBlobKey points to a stored PDF.
- Public verification of revoked certificate must not show valid status.

EXCEPTIONS / HANDLING

PDF rendering failure leaves no issued record. Repeated completion events must not issue duplicate codes or certificates.

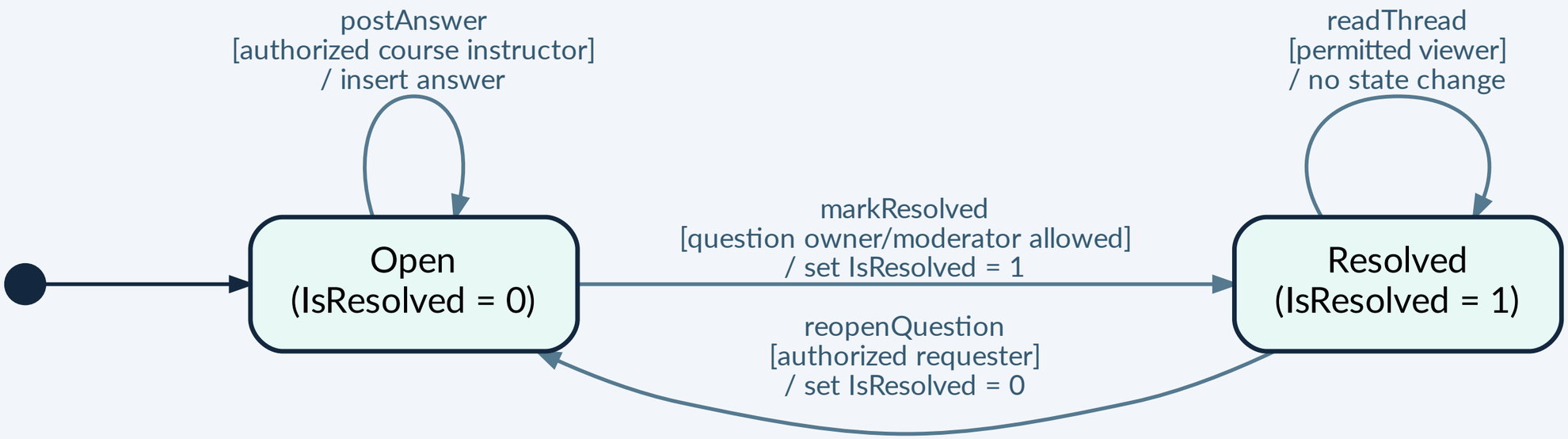
VERIFICATION / TEST

Complete course and issue once; repeat event; verify identical certificate; then revoke and check public verifier returns invalid.

DESIGN NOTE: Revocation is enabled by proposed IsRevoked schema, not stated in the original brief; generating is never stored as a Certificate status.

Course Q&A resolution

SM-11 | ENGAGEMENT | DERIVED / FLAG | CourseQuestions



PERSISTENCE / STATE SOURCE

CourseQuestions.IsResolved

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-11 | transition contract

Learner course questions may receive instructor answers while open. The resolved indicator is a boolean, not a workflow-status enum.

ENTITY / STATE REPRESENTATION

CourseQuestions | CourseQuestions.IsResolved

TRACEABILITY

UC-19, UC-35 / FR-056, FR-057

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Open -> Open	postAnswer	authorized course instructor	insert answer
Open -> Resolved	markResolved	question owner/moderator allowed	set IsResolved = 1
Resolved -> Open	reopenQuestion	authorized requester	set IsResolved = 0
Resolved -> Resolved	readThread	permitted viewer	no state change

INVARIANTS

- Answers are stored in CourseAnswers, not embedded in the question status.
- Instructor ownership/authorization must be checked on replies.
- Resolved does not imply the record has been deleted or hidden.

EXCEPTIONS / HANDLING

Unauthorized answers and malformed text are rejected; moving between states does not remove previous answers.

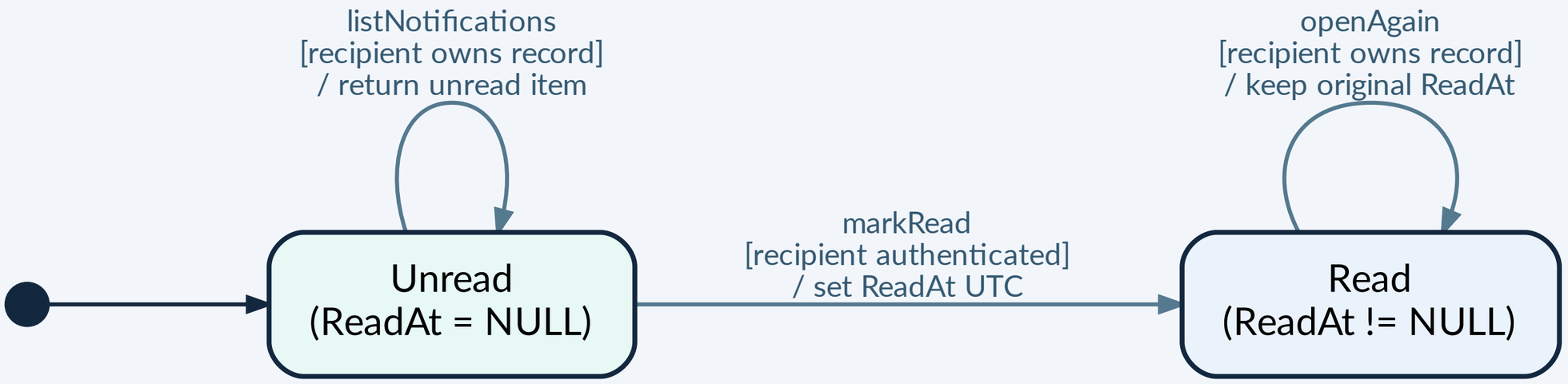
VERIFICATION / TEST

Student opens Q&A; instructor responds; student resolves, reopens; answer history remains.

DESIGN NOTE: Reopen and explicit resolve actions are proposed; brief requires instructor Q&A replies but not a resolution workflow.

In-app notification read status

SM-12 | ENGAGEMENT | DERIVED / TIMESTAMP | Notifications



PERSISTENCE / STATE SOURCE

Notifications.ReadAt (nullable)

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-12 | transition contract

When a notification row is created, its ReadAt is null. Opening the notification sets ReadAt; delivery via external email/push is outside the current schema.

ENTITY / STATE REPRESENTATION

Notifications | Notifications.ReadAt (nullable)

TRACEABILITY

UC-44, UC-45, UC-48 / FR-079

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Unread -> Read	markRead	recipient authenticated	set ReadAt UTC
Unread -> Unread	listNotifications	recipient owns record	return unread item
Read -> Read	openAgain	recipient owns record	keep original ReadAt

INVARIANTS

- Only the recipient can read or mark the record read.
- ReadAt is the sole persistent read-state field.
- Notification creation should follow the originating transaction commit.

EXCEPTIONS / HANDLING

Unsent or failed email notifications are not representable here; if needed, introduce a separate delivery/outbox entity.

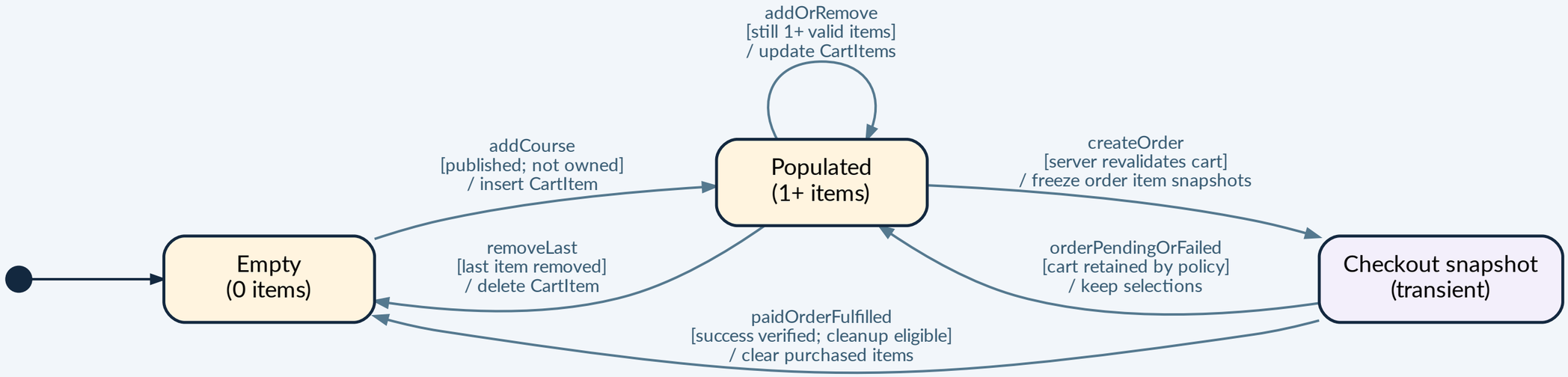
VERIFICATION / TEST

Create notification unread; mark as read; repeat call keeps one timestamp; other user cannot mutate it.

DESIGN NOTE: A notification UI and event triggers are derived from Observer Pattern; external delivery statuses would require a new persistence model.

Shopping cart content lifecycle

SM-13 | CART + CHECKOUT | DERIVED STATE | Carts / CartItems



PERSISTENCE / STATE SOURCE

CartItem count + transient checkout snapshot

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-13 | transition contract

Cart has no status column. Empty and Populated are computed from CartItems; checkout creates a separate immutable Order, not a cart-status value.

ENTITY / STATE REPRESENTATION

Carts / CartItems | CartItem count + transient checkout snapshot

TRACEABILITY

UC-08, UC-10 / FR-045, FR-046, FR-050, FR-053

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Empty -> Populated	addCourse	published; not owned	insert CartItem
Populated -> Populated	addOrRemove	still 1+ valid items	update CartItems
Populated -> Empty	removeLast	last item removed	delete CartItem
Populated -> Checkout	createOrder	server revalidates cart	freeze order item snapshots
Checkout -> Populated	orderPendingOrFailed	cart retained by policy	keep selections
Checkout -> Empty	paidOrderFulfilled	success verified; cleanup eligible	clear purchased items

INVARIANTS

- One Cart per UserId and one CartItem per (CartId, CourseId).
- Order items and prices are immutable snapshots even if cart changes.
- Cart does not grant enrollment access.

EXCEPTIONS / HANDLING

If checkout fails, allow the user to retain cart selections. Changes made during checkout do not rewrite existing order snapshots.

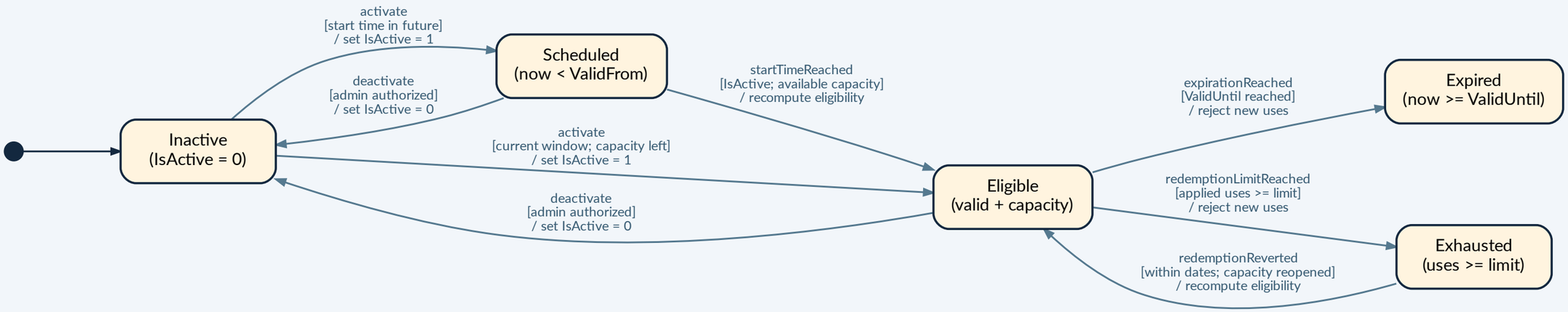
VERIFICATION / TEST

Add/remove items; create checkout order, change cart, settle payment and verify preserved order snapshots.

DESIGN NOTE: Checkout snapshot is transient and cart cleanup is proposed; a physical cart-status enum does not exist.

Coupon eligibility evaluation

SM-14 | PRICING + DISCOUNTS | COMPUTED POLICY STATE | Coupons



PERSISTENCE / STATE SOURCE

Coupons.IsActive, ValidFrom, ValidUntil, MaxRedemptions

SEMANTICS

Guards and effects are proposed unless explicitly stated in the brief.

SM-14 | transition contract

Coupon eligibility is derived at evaluation time from activation, dates, and applied-redemption counts. It is not stored as a Coupon.Status enum.

ENTITY / STATE REPRESENTATION

Coupons | Coupons.IsActive, ValidFrom, ValidUntil, MaxRedemptions

TRACEABILITY

UC-09, UC-29 / FR-047, FR-048

STATE TRANSITIONS

FROM -> TO	TRIGGER	GUARD / CONDITION	PERSISTENCE / EFFECT
Inactive -> Scheduled	activate	start time in future	set IsActive = 1
Inactive -> Eligible	activate	current window; capacity left	set IsActive = 1
Scheduled -> Eligible	startTimeReached	IsActive; available capacity	recompute eligibility
Eligible -> Exhausted	redemptionLimitReached	applied uses >= limit	reject new uses
Eligible -> Expired	expirationReached	ValidUntil reached	reject new uses
Eligible -> Inactive	deactivate	admin authorized	set IsActive = 0
Scheduled -> Inactive	deactivate	admin authorized	set IsActive = 0
Exhausted -> Eligible	redemptionReverted	within dates; capacity reopened	recompute eligibility

INVARIANTS

- Eligibility is evaluated on the server at checkout, not trusted from the client.
- Only Applied redemptions consume the usage limit.
- DiscountValue follows Percentage/Fixed constraints in the schema.

EXCEPTIONS / HANDLING

A time-based state may change without a database update. Reactivation after expiry still results in Expired unless validity dates are changed.

VERIFICATION / TEST

Evaluate before/after start and end time; reach MaxRedemptions, revert applied use and verify eligibility recalculation.

DESIGN NOTE: These are mutually selected policy outcomes, not literal persisted states. Precedence: Inactive > Expired > Scheduled > Exhausted > Eligible.

Cross-object state invariants

Publishing

Courses must be Published before browse, enrollment or streaming is authorized.

Paid access

Order Paid and Payment Succeeded require server-verified events; enrollment fulfillment is idempotent.

Media safety

Media Ready gates protected playback. Do not store or expose permanent signed URLs.

Completion

Enrollment Completed is derived from the required lessons and quizzes; never infer from video playback alone.

Certificates

One certificate per enrollment; issuance after completion; revocation affects verification.

Discounts

Coupon eligibility must be re-evaluated on server; only Applied redemptions count toward limits.

Auditing

Webhook processing state is independent of payment state; event uniqueness avoids repeated effects.

Privacy

Only the owner can see personal progress/notifications; public verification exposes minimal credential data.

Consistency with the physical schema

Do not create database statuses from visual-only states: the ERD/schema baseline already specifies exact persistable columns.

ENUM VALUES

Course / Media / Enrollment / QuizAttempt / Order / PaymentTransaction / PaymentWebhookEvent / CouponRedemption.

FLAGS & TIMESTAMPS

LessonProgress.IsCompleted, Certificate.IsRevoked, CourseQuestion.IsResolved, Notification.ReadAt.

ROW EXISTENCE / COUNT

Not-started lesson: no progress row. Cart Empty/Populated: count CartItems. Certificate NotIssued: no certificate row.

COMPUTED ELIGIBILITY

Coupon Scheduled, Eligible, Expired and Exhausted derive from dates, IsActive and Applied redemptions.

TRANSIENT OPERATIONS

Video Uploading, certificate Generating and cart CheckoutSnapshot are actions/work-in-flight, not persisted status fields.

Test coverage and event safety

01	**Access control**	Unauthorized role and owner attempts cannot trigger state changes.
02	**Repeatability**	Repeated user commands, webhooks and completion notifications do not duplicate effects.
03	**Ordering**	Out-of-order provider events never regress Succeeded to Failed or Paid to Pending.
04	**Concurrency**	Two students checkout independently; duplicate enrollment insert conflicts are handled; course RowVersion prevents lost updates.
05	**Failure / retry**	Processing failures remain identifiable; retries return only along explicitly allowed edges.
06	**Data invariants**	SQL CHECK constraints and unique indexes reject forbidden statuses, duplicate records and invalid numeric ranges.
07	**Observability**	Logs correlate order, payment transaction, webhook event and enrichment jobs without leaking secrets.
08	**Acceptance**	Each SM machine is tested for nominal path, failed guard, repeated trigger and at least one exception.

Decisions requiring team approval

01 **Course release**

The brief supports immediate publishing, while the admin panel includes course approval. This model requires approval before public release.

02 **Course edits**

Specify how changes to an already-Published course are re-reviewed or versioned without affecting current students.

03 **Refund behavior**

Define full versus partial refunds, enrollment revocation, access after revocation and order reconciliation.

04 **Quiz attempts**

Choose retake limits, unfinished-attempt timeout policy, and handling of partially completed quizzes.

05 **Coupon policy**

Define reservation timing, expiry during checkout, usage-limit locking and coupon reactivation.

06 **Operational workers**

Choose durable retry infrastructure for Blob encoding, certificate generation and notification delivery.

References & review checklist

S1 EduHub_ProjectDetails(1).pdf, pp. 1-5 - original team project brief.

S2 EduHub - ER Diagram (previous deliverable) - 38-entity candidate model.

S3 EduHub - Logical & Physical Schema (previous deliverable) - SQL status and flag baseline.

S4 EduHub - Use Case Diagram & Descriptions, Functional Requirements, Sequence and Activity Diagrams.

R1 Object Management Group, UML 2.5.1: https://www.omg.org/spec/UML/2.5.1/

R2 Microsoft EF Core concurrency tokens: https://learn.microsoft.com/en-us/ef/core/modeling/concurrency

R3 Stripe payment fulfillment: https://docs.stripe.com/checkout/fulfillment

R4 Microsoft Azure Storage SAS: https://learn.microsoft.com/en-us/azure/storage/common/storage-sas-overview

REVIEW BEFORE IMPLEMENTATION

Confirm event/guard rules, whether state is persisted or derived, allowed terminal transitions, concurrency handling, traceability to use cases, and changes that would require a database migration.